CIS*3110 Operating Systems

Week 13
Lecture #22 LFS and SSD

Prof. Yulia Kotseruba
April 2, 2026

Quiz ??

- 3 random hard questions:
  - Which process has a PID of 1? a) any kernel process b) orphan process c) zombie process d) none of the above
  - In CFS, a process with lower nice value gets a) less CPU time b) more CPU time c) more vruntime d) higher priority
  - What is a major disadvantage of swapping? a) uses too much CPU b) slow due to I/O c) fragmentation d) TLB misses

Recap

- **Crash consistency**
  - vsfs and FFS have poor crash consistency
  - crashes can cause metadata issues and data loss
- **fsck - a passive solution**
  - scan the hard-drive and rebuild the file system
  - cons: takes a very long time to do
- **journaling - an active solution**
  - write data/metadata to a log first
  - pros: don’t need to scan the whole hard drive

Recap

- **Journal write** - write the transaction to the journal
  - transaction-begin block
  - pending metadata and data updates
  - wait for the writes to complete

if crash happens before the transaction is fully journaled -> the update is invalid and will be skipped

no data consistency problems
data loss

- **Journal commit**
  - write transaction-end block
  - wait for the writes to complete
- **Checkpoint**
  - write the pending metadata and data updates to the disk

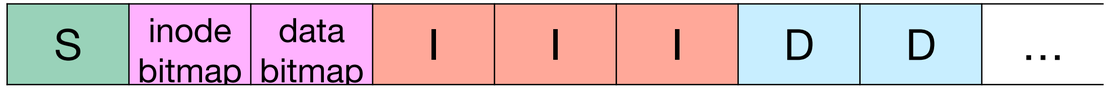
if crash happens during checkpoint -> redo the journaled operations during boot

no data consistency problems
no data loss

- **Free**
  - mark the transaction free in the journal superblock

Finding an inode

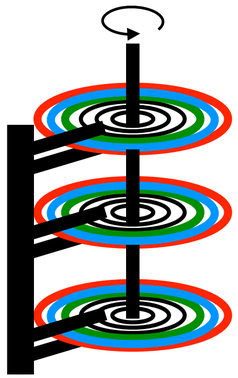
vsfs



superblock stores location of the inode table

inode is an offset into inode table

data blocks are linked from inodes



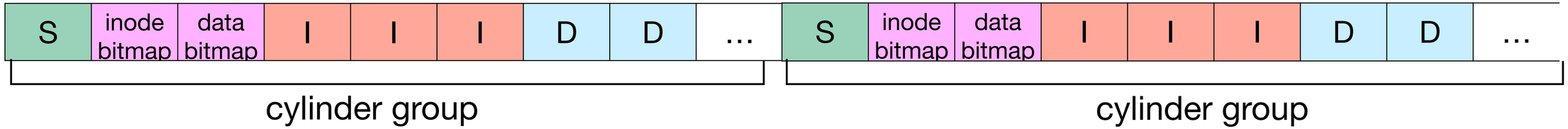
FFS duplicates vsfs structure in every cylinder group

Superblock at the beginning of each group stores group data

if inode is not in this group, go to the next group

the rest is the same

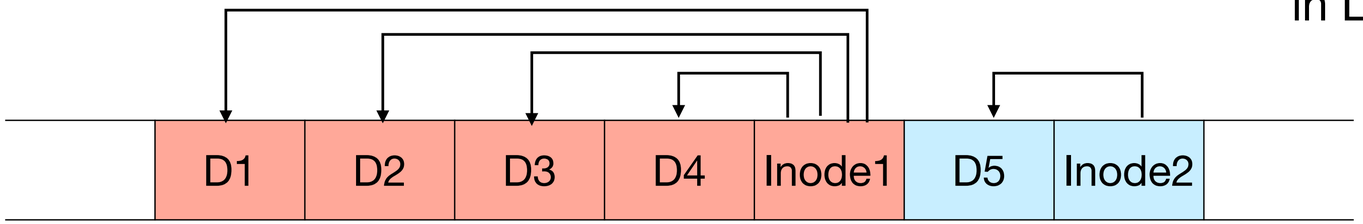
FFS



in LFS inodes are scattered across the drive

Problem: how can we find an inode for any file without scanning the whole drive?

LFS

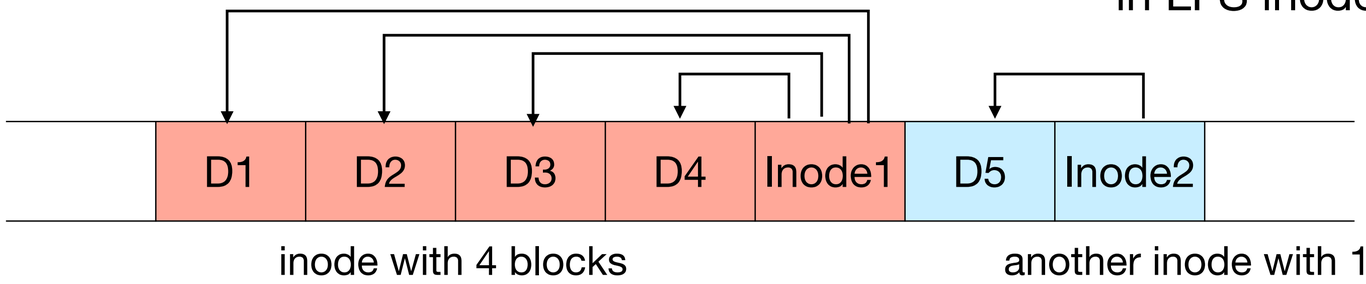


inode with 4 blocks

another inode with 1 block

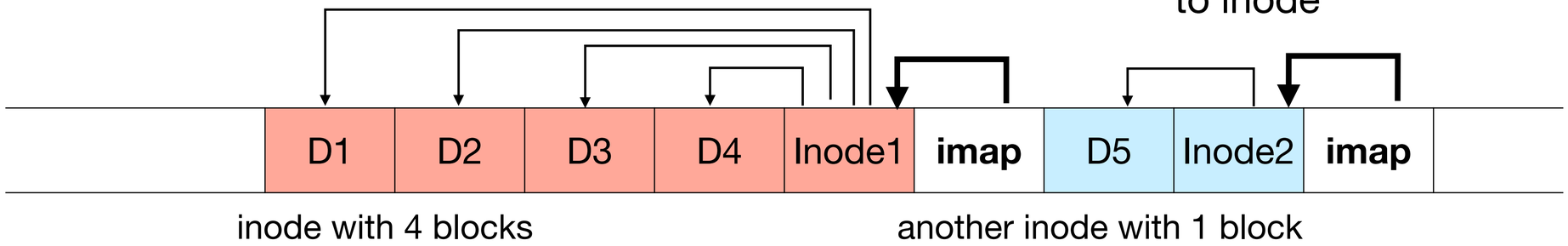
LFS inode map

in LFS inodes are scattered across the drive



Problem: how can we find an inode for any file without scanning the whole drive?

Solution: write an inode map with the link to inode

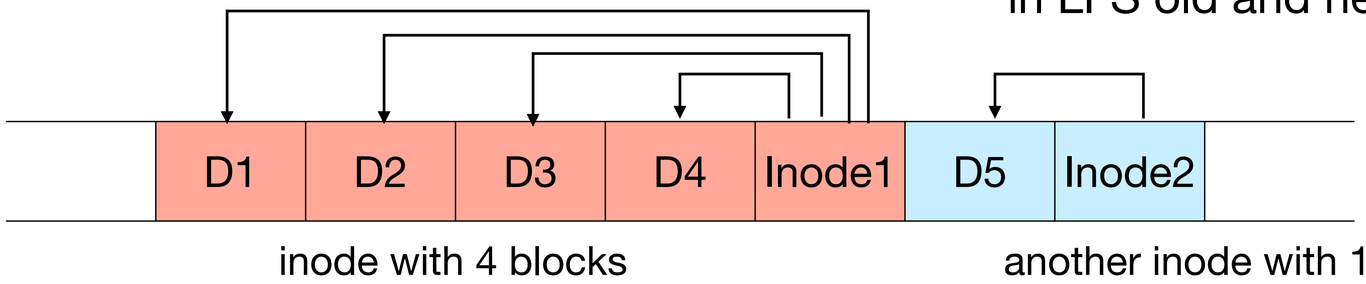


Problem: how can we find the imaps?

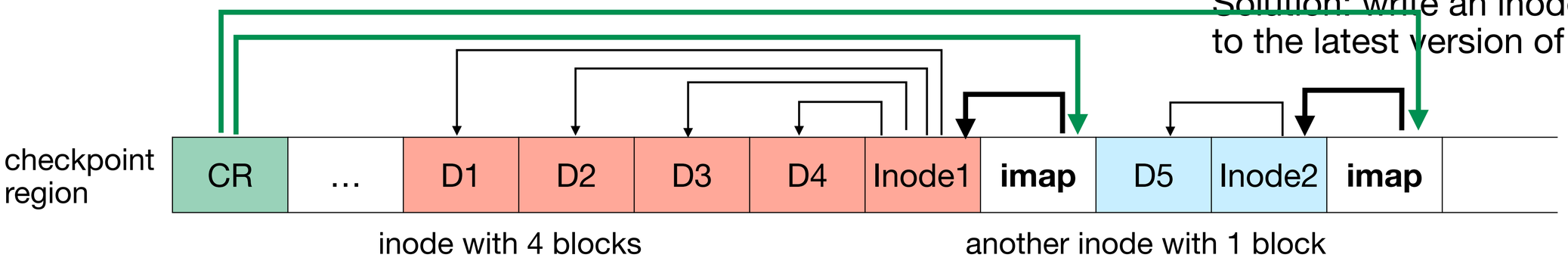
LFS inode map

in LFS old and new inodes are scattered across the drive

Problem: how can we find an inode for any file without scanning the whole drive?



Solution: write an inode map with the link to the latest version of inode



Problem: how can we find the imaps?

Solution: add a checkpoint region (CR) at a fixed location on the disk with links to the pieces of inode map

LFS inode map

- **Checkpoint region** at a fixed location on the drive
  - contains **pointers** to the pieces of inode map
    - a system-wide table of all files and directories
  - inode map points to the most recent version of inode
  - inode points to the data
    - via direct/indirect pointers
- **Checkpoint region is updated rarely** (every 5-30 s) to reduce I/O overhead

LFS finding an inode

- Read the checkpoint region
  - Access all imaps and cache them in RAM
- Look up inode in imap
- Follow the link to the most recent version
- Follow the block links (direct or indirect) from inode to its data

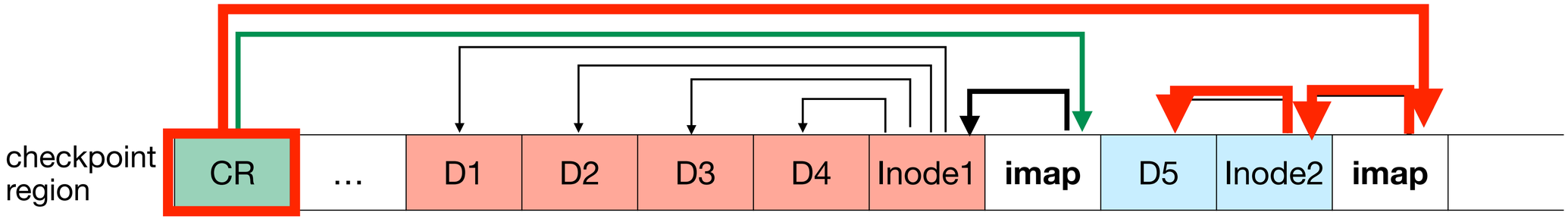
accessing inode2

cache CR in RAM

find imap that has inode2

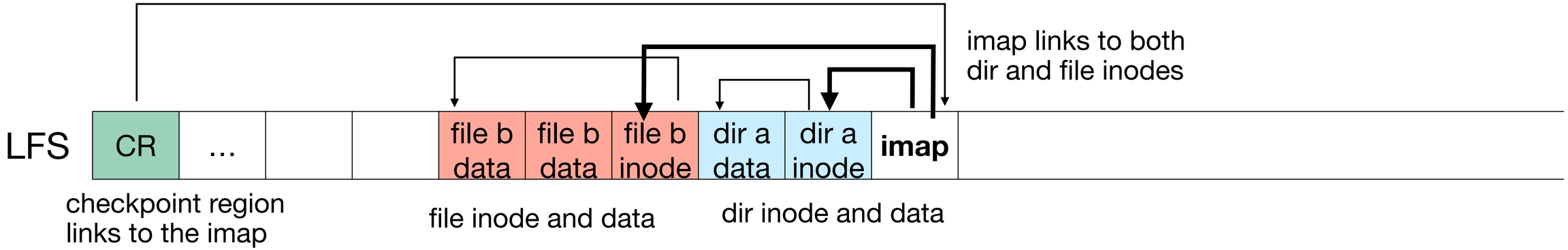
follow the link from imap to inode2

follow the link from inode2 to data



LFS directories

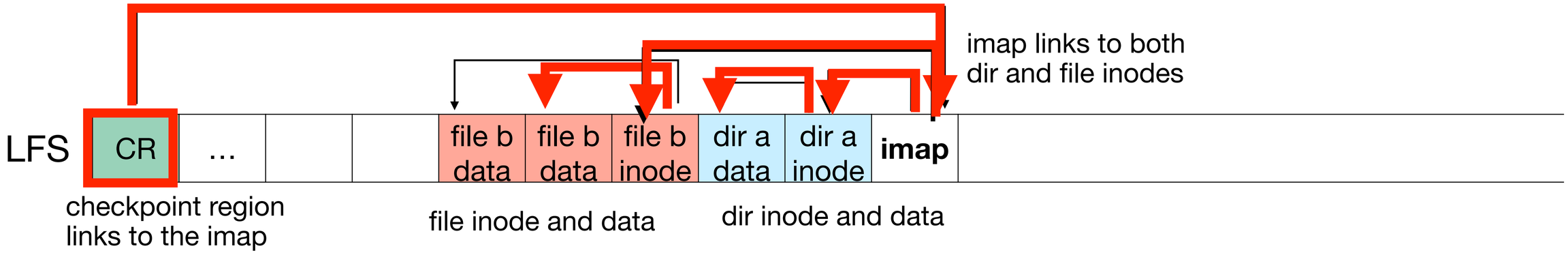
- **Directories are treated same way as files**
  - Imap points to the directory inode
  - Directory inode points to directory data
  - Directory data lists names and inodes for files in the directory



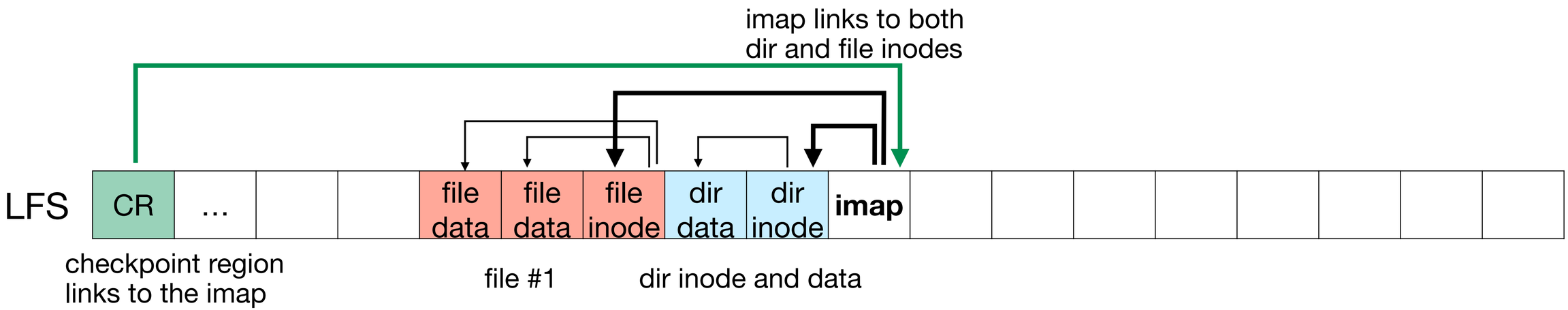
LFS directories

- **Directories are treated same way as files**
  - Imap points to the directory inode
  - Directory inode points to directory data
  - Directory data lists names and inodes for files in the directory

accessing file a/b
cache CR in RAM
split the path into a and b
find inode of a in RAM
follow the link from imap to inode a
follow the link to a data
find inode of b in the directory a data
find inode of b in RAM
follow the link from imap to inode b
follow the link from inode b to its data

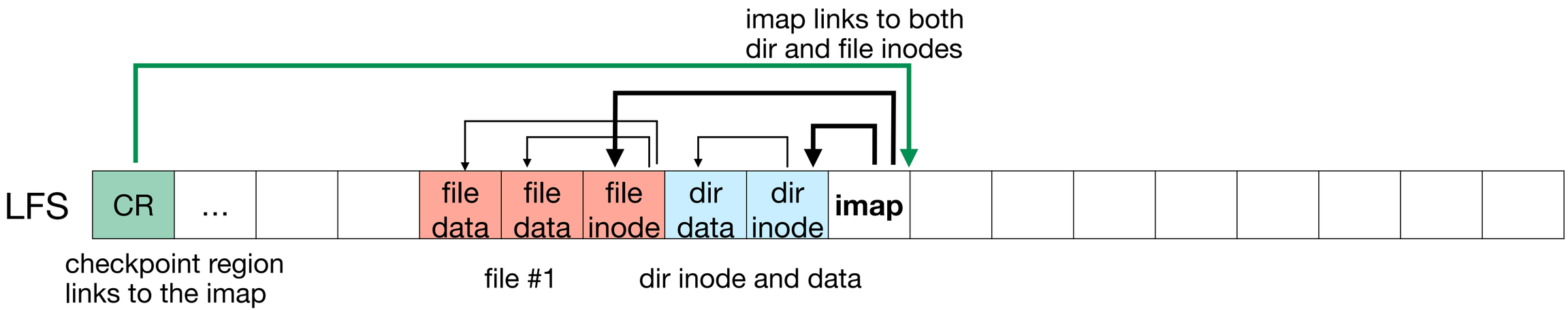


LFS directories



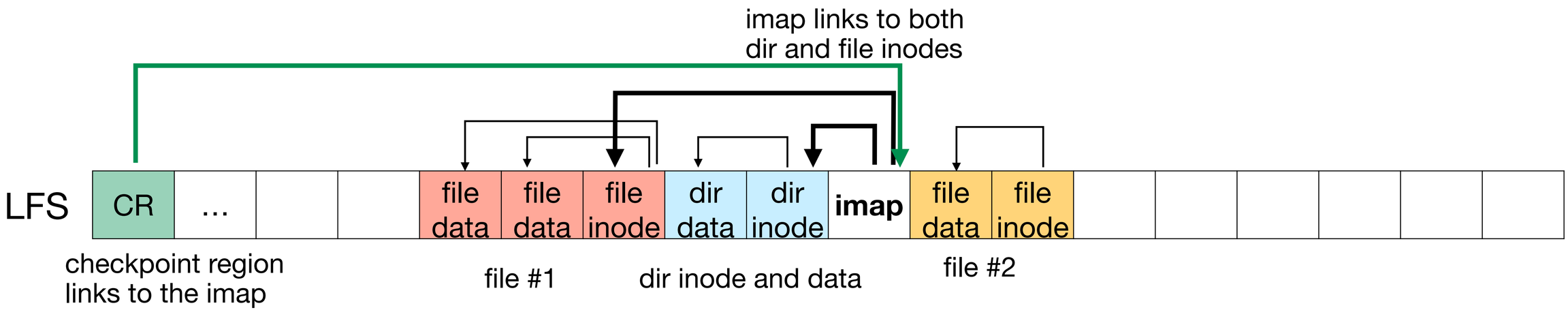
what if we add another file to the directory?

LFS directories



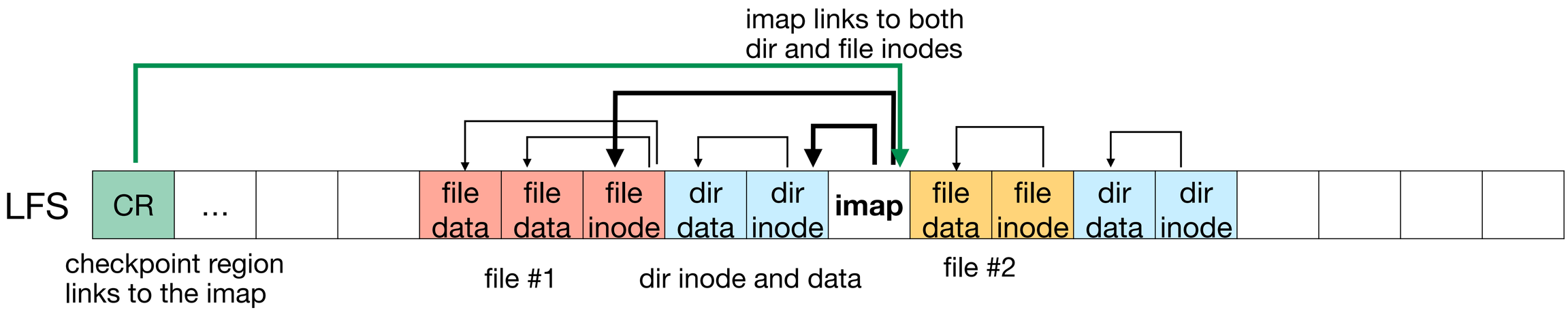
what if we add another file to the directory?

LFS directories



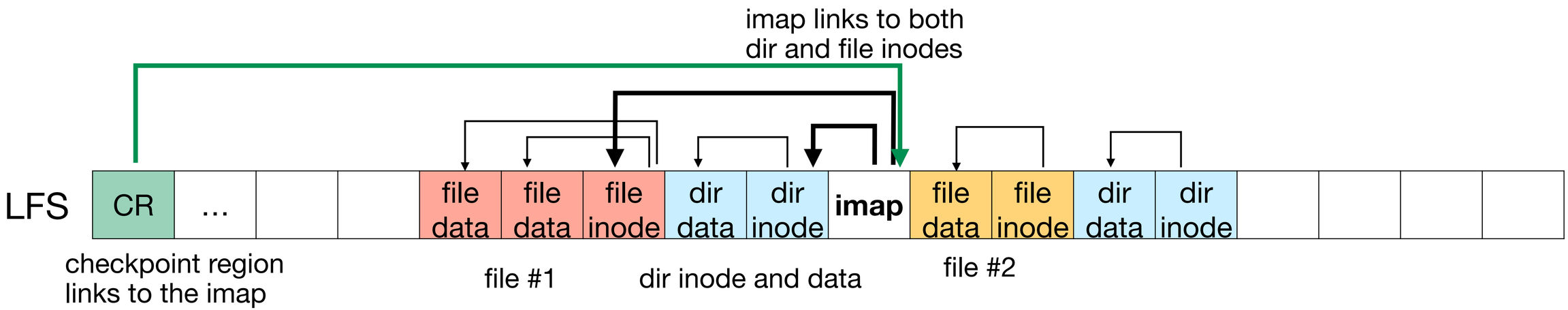
what if we add another file to the directory?
write file #2 and update dir inode

LFS directories



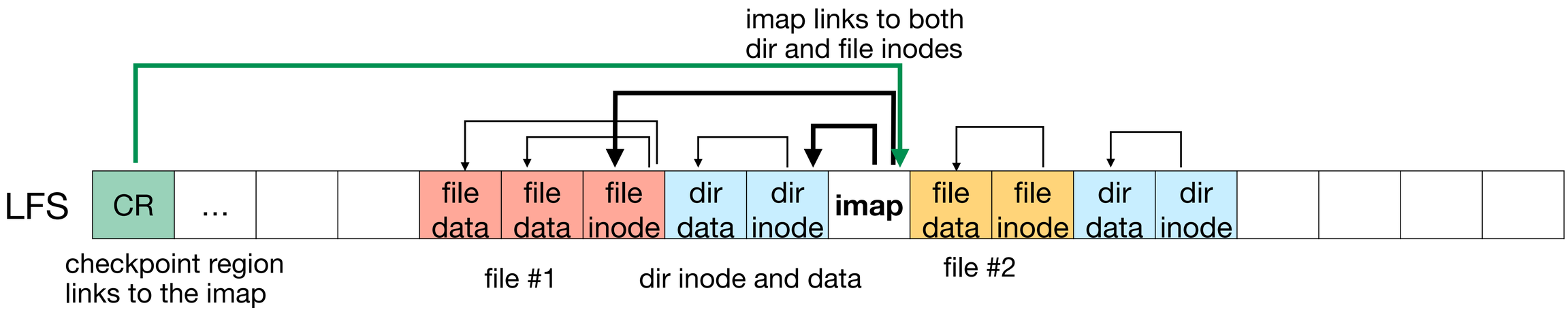
what if we add another file to the directory?
add inode for file#2 to dir data
write file #2 and updated dir inode

LFS directories



what if we add another file to the directory?
add inode for file#2 to dir data
write file #2 and updated dir inode

LFS directories



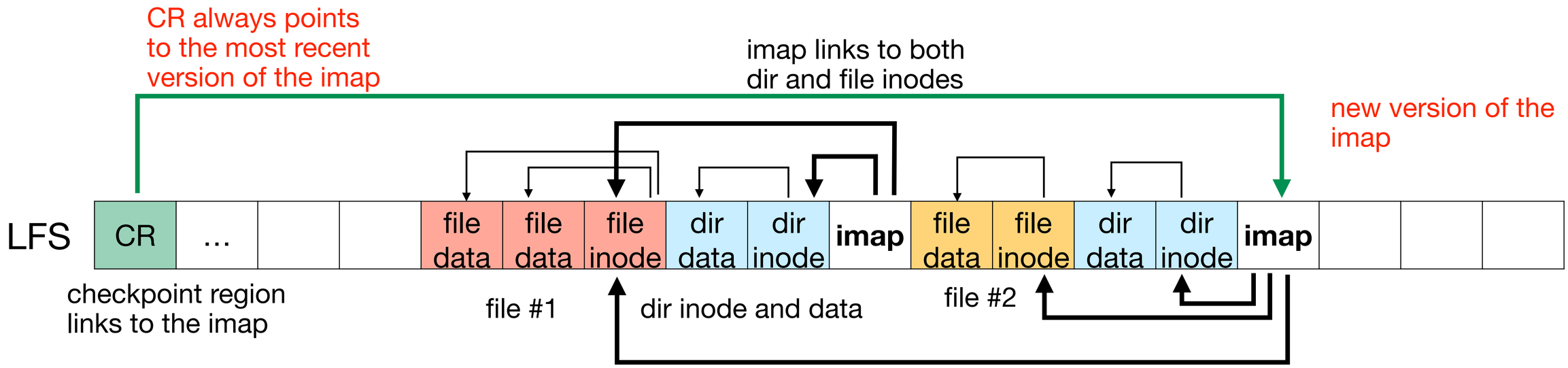
what if we add another file to the directory?

add inode for file#2 to dir data

write file #2 and updated dir inode

inode for file #1 is the same

LFS directories



what if we add another file to the directory?

add inode for file#2 to dir data

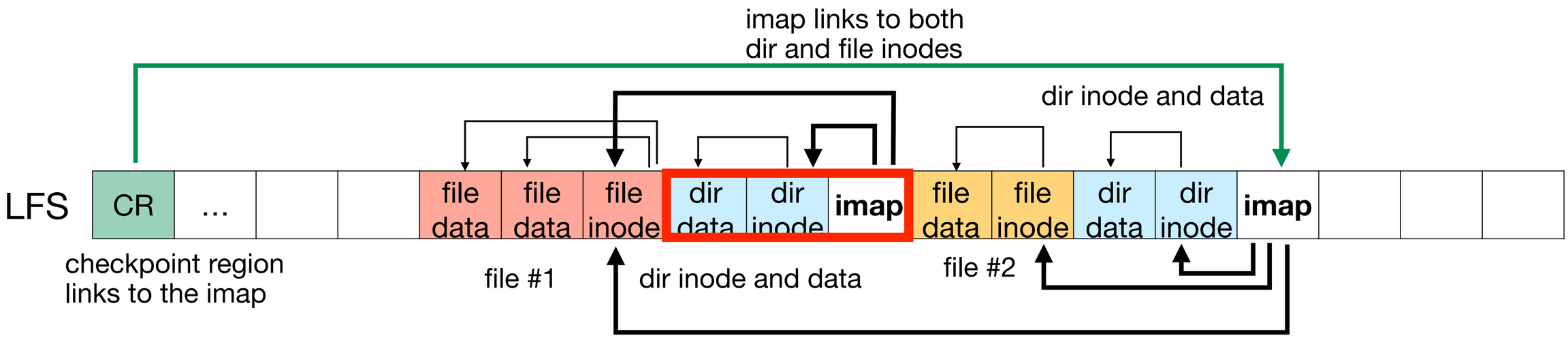
write file #2 and updated dir inode

inode for file #1 is the same

imap points to the new dir inode, file #1 inode and file #2 inode

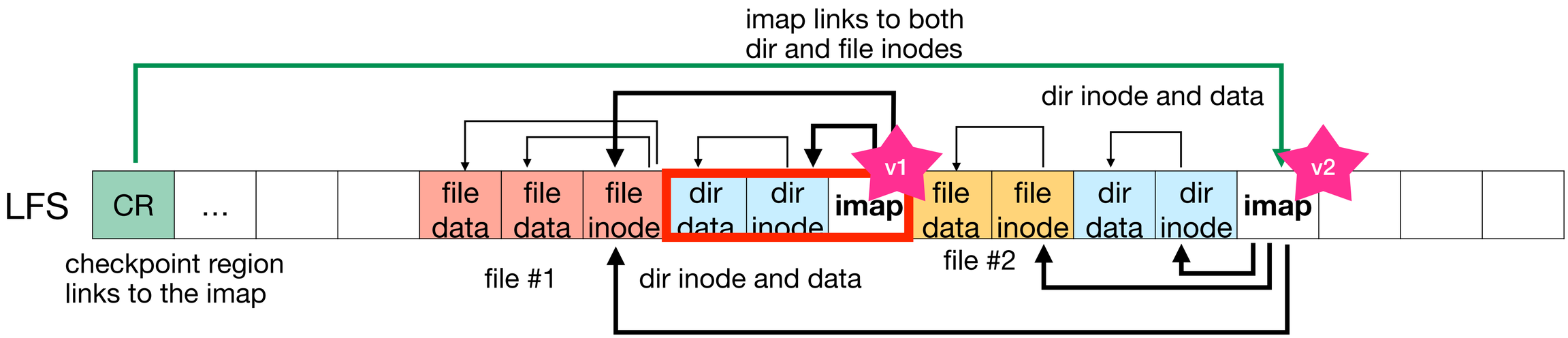
CR points to the updated version of the imap

Garbage collection



What do we do with the "old" data on HD?

Garbage collection

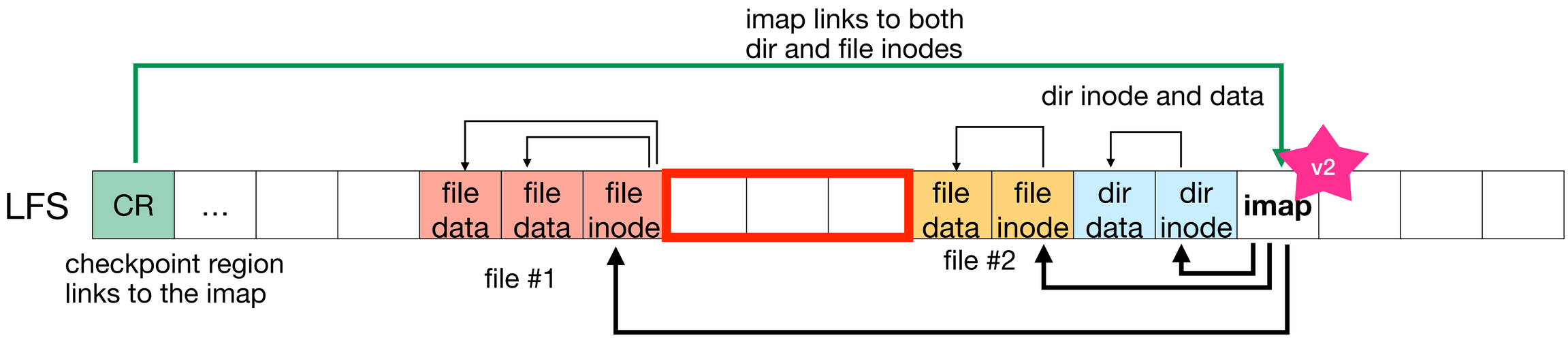


What do we do with the "old" data on HD?

Solution #1: assign versions to data and keep them as snapshots (aka versioning file systems)

Solution #2: remove old data (this is what LFS does)

Garbage collection

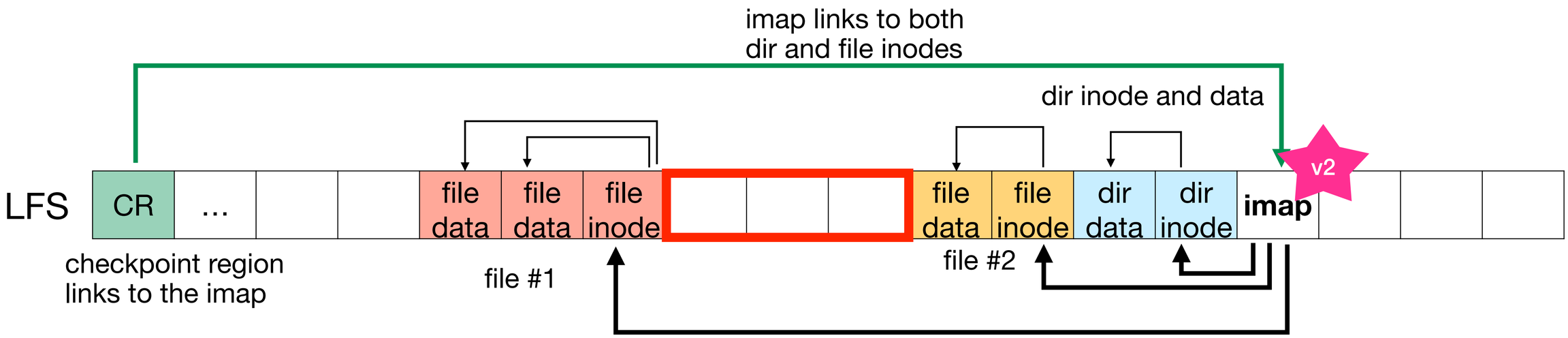


What do we do with the "old" data on HD?

~~Solution #1: assign versions to data and keep them as snapshots (aka versioning file systems)~~

Solution #2: remove old data (this is what LFS does)

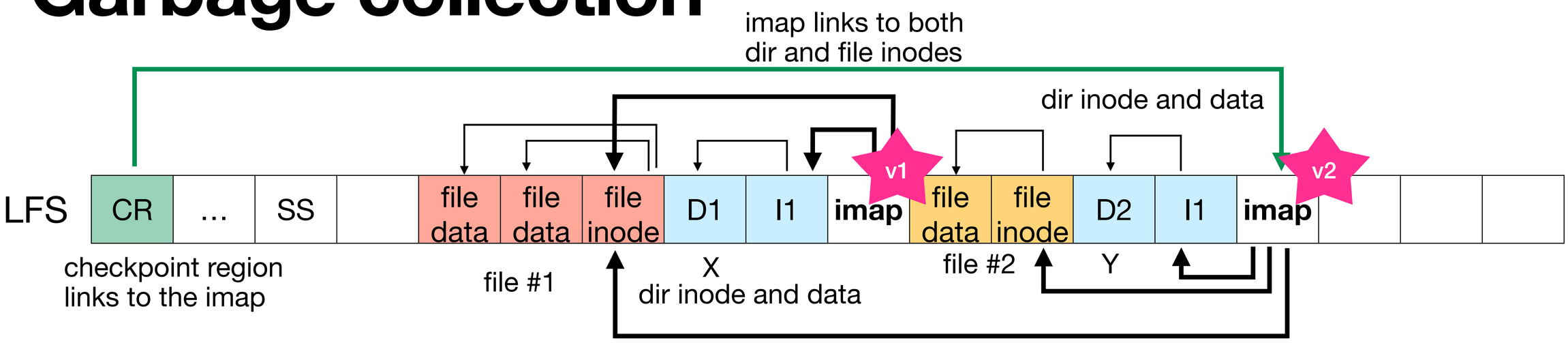
Garbage collection



How do we know which blocks are OK to remove?

What do we do with external fragmentation?

Garbage collection



How to check if the block is dead/alive?

start with D1

A = X

which file it belongs to

which block of the file this is

segment summary block			
block addr (A)	data block	inode # (N)	offset (T)
X	D1	I1v1	1
Y	D2	I1v2	1
...	...	...	...

```
(N, T) = SegmentSummary[A]
inode = Read(imap[N])
if (inode[T] == A)
// block D is alive
else
// block D is garbage
```

inode=I1v1 offset=1

inode=I1v2

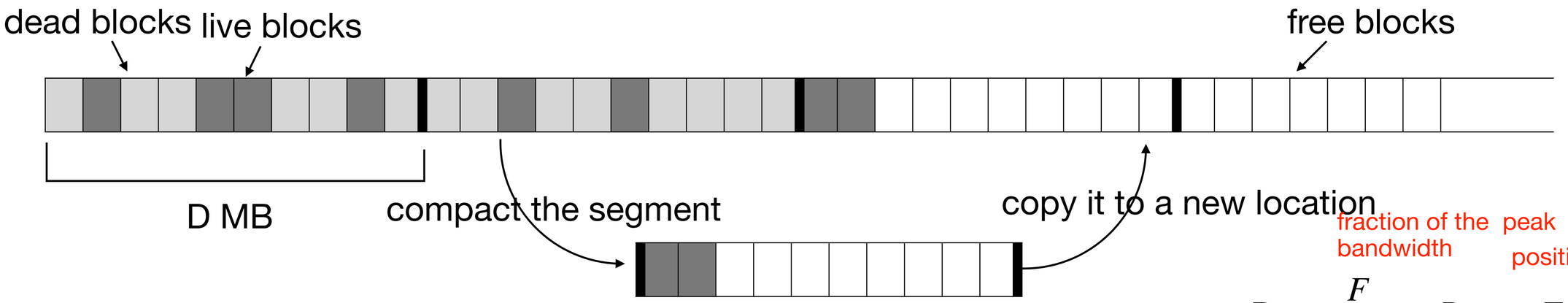
I1v2 points to Y

SS points to X

block D1 is dead

a list of written blocks with additional information

Garbage collection



fraction of the peak bandwidth

positioning latency

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position}$$

segment size

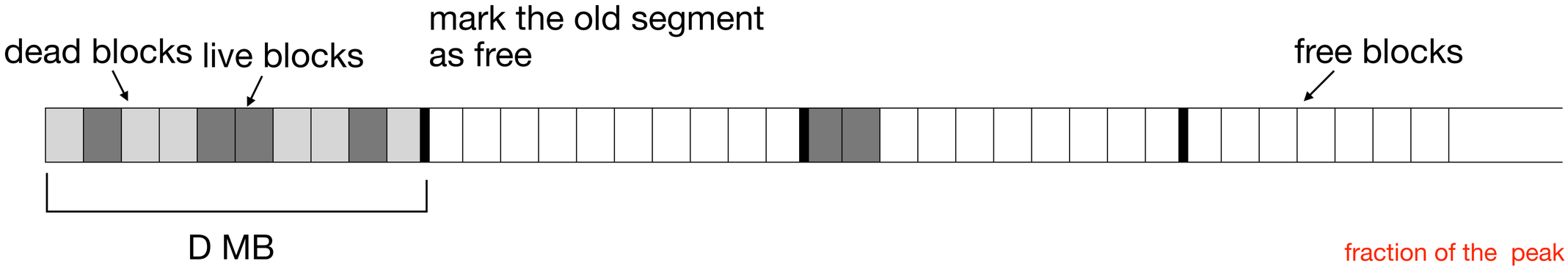
peak transfer rate

Writes are done in segments of size D

Scan segments and count dead blocks

If # dead blocks > threshold -> compact and write into a new segment

Garbage collection



fraction of the bandwidth

peak positioning latency

$$D = \frac{F}{1 - F} \times R_{peak} \times T_{position}$$

segment size

peak transfer rate

Writes are done in segments of size D

Scan segments and count dead blocks

If # dead blocks > threshold -> compact and write into a new segment

mark the old segment as free to be overwritten

Garbage collection

- **Scan disk segments periodically**
  - Identify dead and alive blocks
    - use segment summary block to trace references
      - alive block - block data address in imap and segment summary match
      - dead block - otherwise
  - If number of dead blocks > threshold
    - compact the segment and write it in free space
    - mark the "old" segment as free for reuse

Crash recovery

- File system consistency
  - **checkpoint** - records the last consistent state of the file system
    - saved **every 30 sec atomically**
      - write a timestamp
      - write the pointers to imaps
      - write the end timestamp
      - if the last timestamp is missing, checkpoint is invalid
- Data loss - at most 30 s

LFS summary

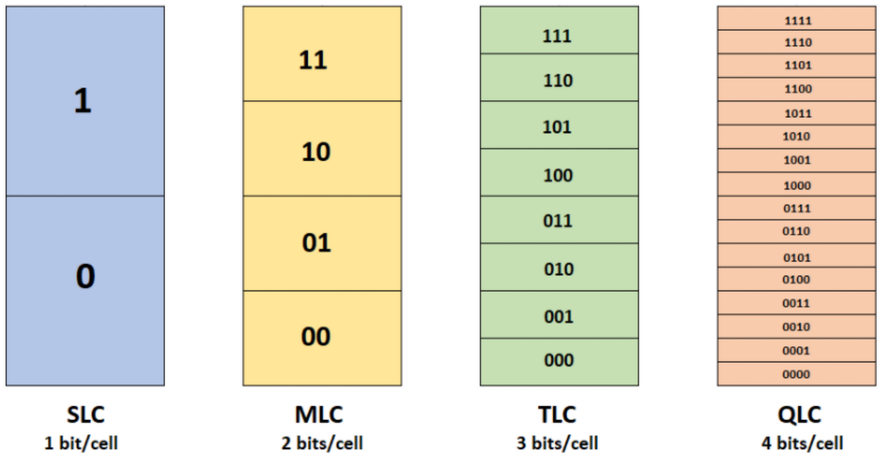
- **Log-structured file system (LFS)**
  - improves both write efficiency and crash recovery
  - the whole file system is a log -> no in-place writes!
    - most reads are cached
  - updates the drive every 30 s
    - at most 30 s data loss

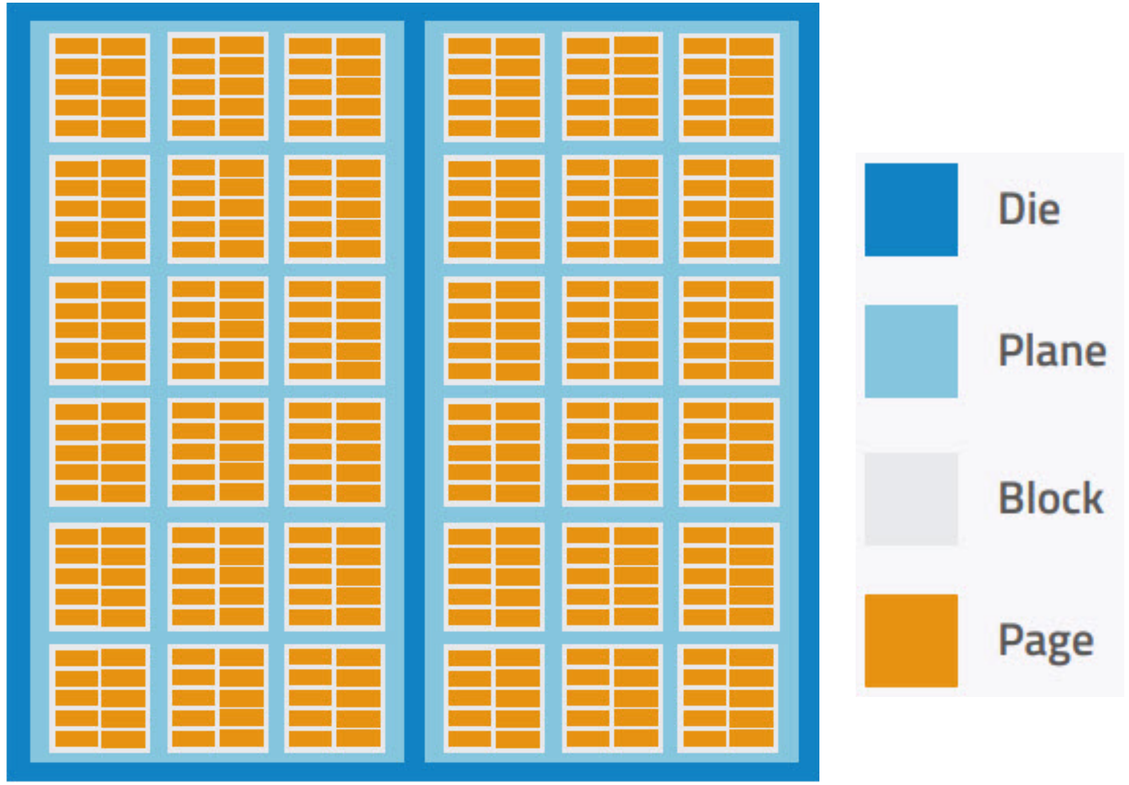
SSD

- Solid-state storage device (introduced in 1990s)
  - non-volatile semiconductor memory
  - based on NAND-based flash (1980s)
- SSDs
  - pros:
    - do not have any moving parts
  - cons:
    - cannot overwrite existing data
    - writing wears them out

Flash memory

- Cell - a single transistor
  - Single-level cell (SLC)
    - holds 1 bit of data
  - Multi-, triple, and quad-level cells hold 2, 3, and 4 bits
- Page - 8 KB
- Block - 1 MB (128 pages)
- Plane - 4 GB (4096 blocks)
- Die - 8 GB (2 planes)

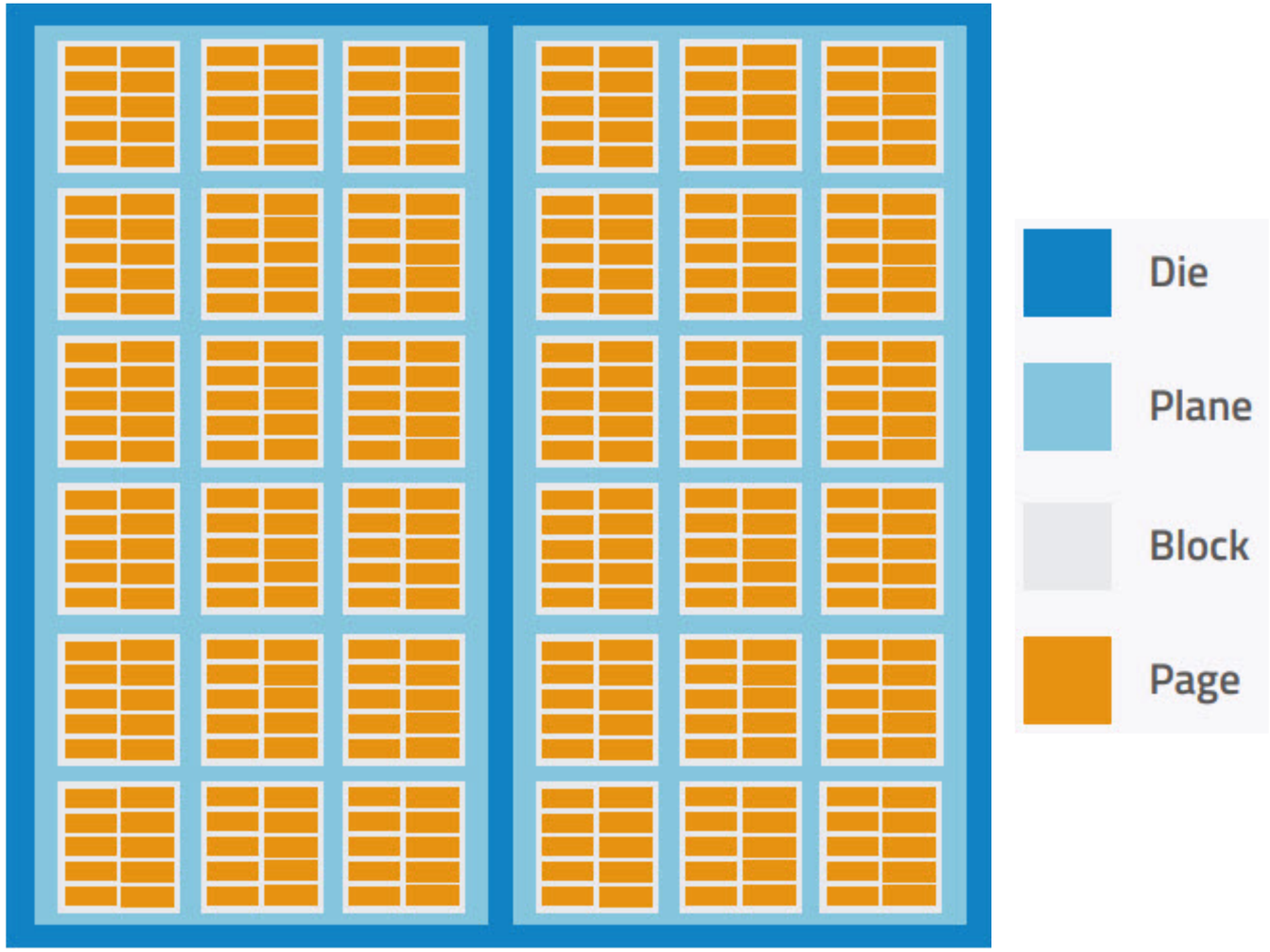




Flash memory

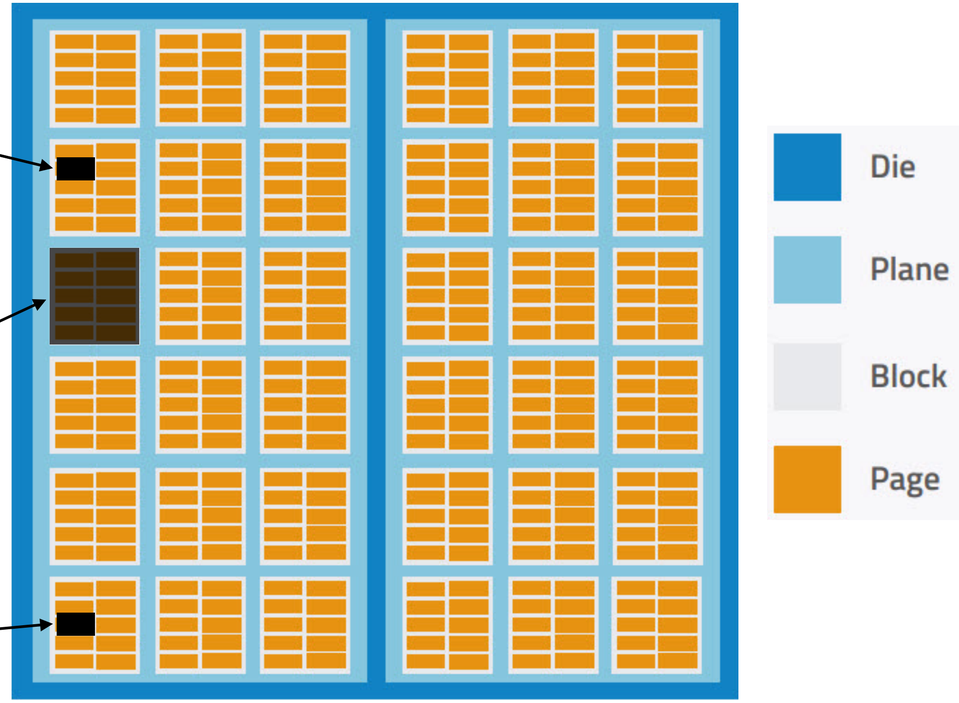
- Cell - a single transistor
  - Single-level cell (SLC)
    - holds 1 bit of data
  - Multi- and triple-level cells hold 2 and 3 bits
- Page - 8 KB
- Block - 1 MB (128 pages)
- Plane - 4 GB (4096 blocks)
- Die - 8 GB (2 planes)
- Bank - 16 GB (2 dies)
- SSD drive - 1 TB (64 banks)

bank



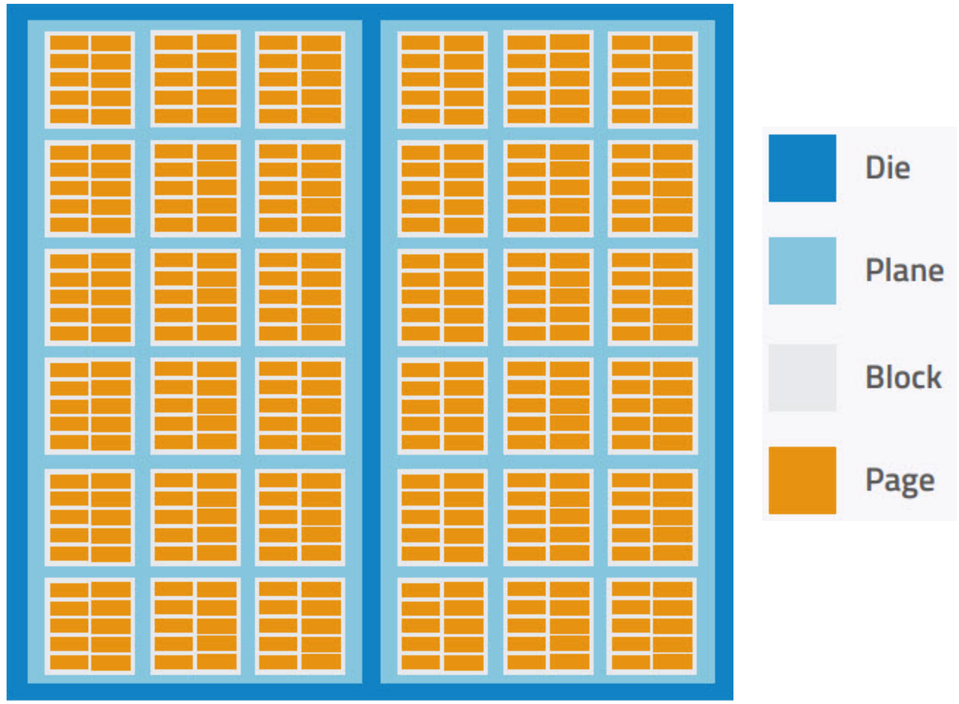
Flash operations

- Read
  - can read only pages (i.e. 8KB at a time)
  - ~10 microsecond latency in any location
- Erase
  - can only erase a block (i.e. 1MB at a time)
  - sets every bit to 1
- Program
  - can only program a page
  - changes some 1's to 0's
  - must be preceded by erase



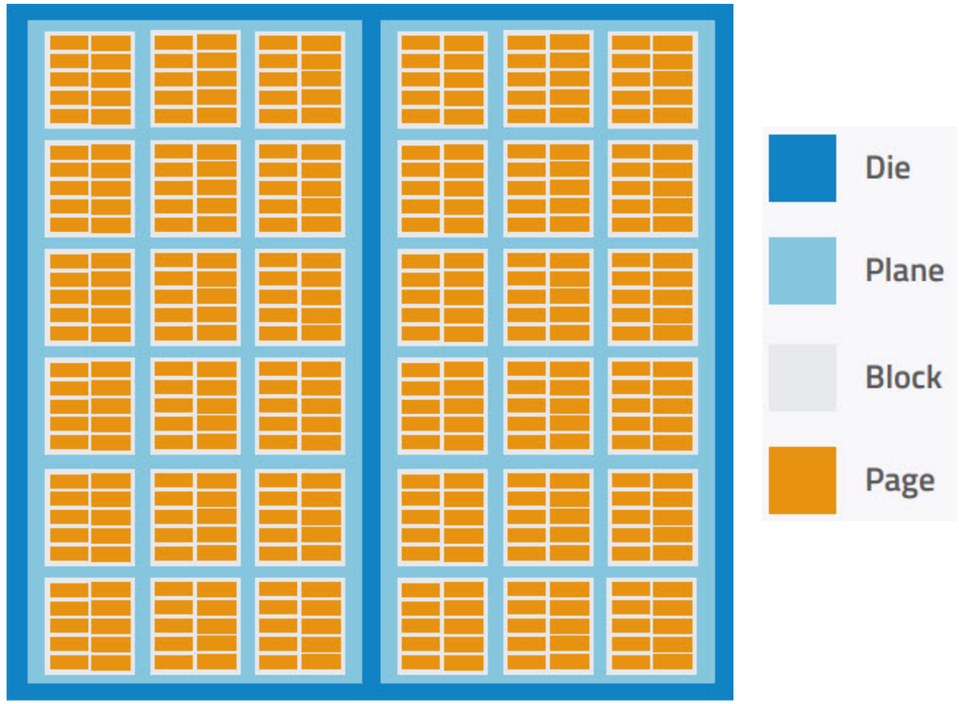
Flash operations

- Reads are fast and similar to RAM
- Writes are tricky
  - **no in-place writes**
  - must erase a block of data first
  - then reprogram pages



Page states

- INVALID - the initial state
- ERASED - programmable state
  - all bits in the block reset to 1
- VALID - programmed, can be read now
- Reads do not affect states
- The only way to write is to erase and program



A toy example

1 block with 4 pages and 8 bits in each page

Block 0			
Page 0	Page 1	Page 2	Page 3
????????	????????	????????	????????
INVALID	INVALID	INVALID	INVALID

The initial state is INVALID

A toy example

1 block with 4 pages and 8 bits in each page

Block 0			
Page 0	Page 1	Page 2	Page 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

The initial state is INVALID

Needs to be erased first to be able to program the block

A toy example

even if you only want to write 1 bit, you still need to update the whole page

Write something to page 0

It will change status to VALID

Any subsequent writes to page 0 will fail -> it is now read-only

Block 0

Page 0	Page 1	Page 2	Page 3
11000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

Now write something in the rest of the pages

Page 0	Page 1	Page 2	Page 3
11000011	**11001110**	**10000010**	**11001111**
VALID	**VALID**	**VALID**	**VALID**

Now overwrite page 0 with something else

To do that, need to erase the whole block

Page 0	Page 1	Page 2	Page 3
11111111	11111111	11111111	11111111
ERASED	ERASED	ERASED	ERASED

We can now write to page 0

But data in pages 1,2 and 3 are gone :(

Should have moved that data somewhere first

Page 0	Page 1	Page 2	Page 3
00000011	11111111	11111111	11111111
VALID	ERASED	ERASED	ERASED

Flash performance

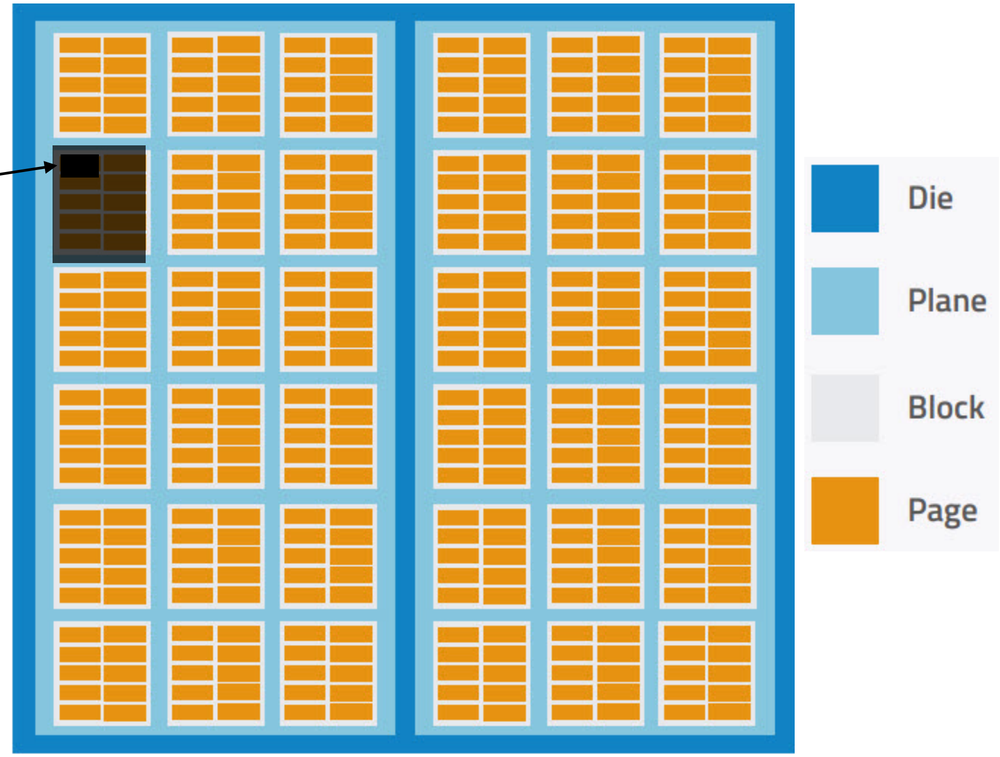
- SLC is the simplest, most reliable and most expensive cell
- QLC has the highest storage density (4x of SLC), is the cheapest and least reliable
- Most SSDs use TLC or QLC
  - sometimes mixed with SLC
- Reads/writes are very fast
- All cells wear out from repeated writing/erasing
  - limited number of Program/Erase cycles

Device	Read (μs)	Program (μs)	Erase (μs)
SLC	25	200-300	1500-2000
MLC	50	600-900	~3000
TLC	~75	~900-1350	~4500

Device	P/E cycles
SLC	50K-100K
MLC	3K-10K
TLC	1K-3K
QLC	100-1000

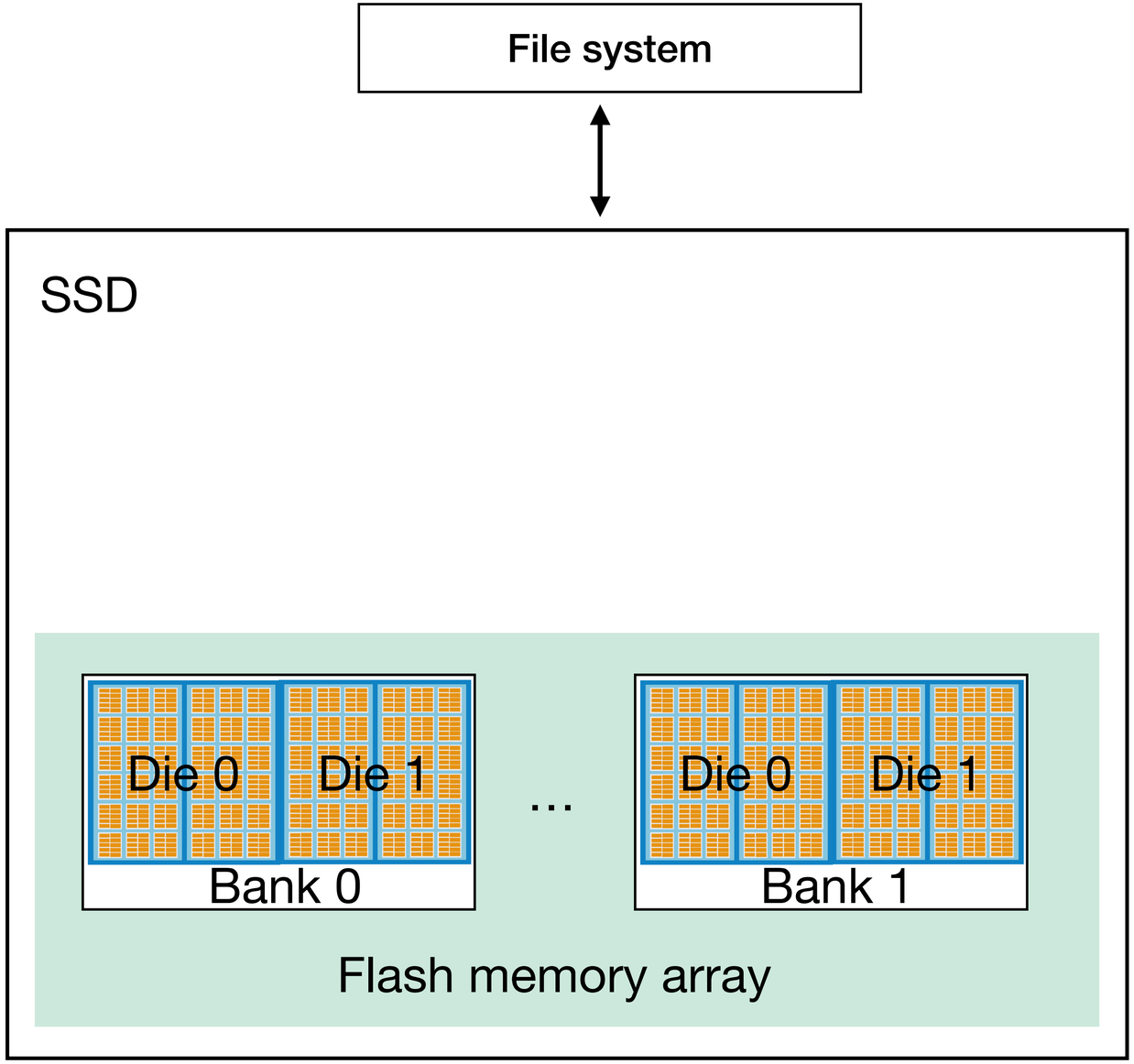
Summary so far

- Flash memory has **no moving parts**
  - reads behave like RAM (no positioning cost)
- **cannot overwrite** existing data
  - can write to 1 page only after erasing it first
  - but erasing works on a block level
  - need to move pages we want to keep to another location first
- erasing/programming **wears out** the cells



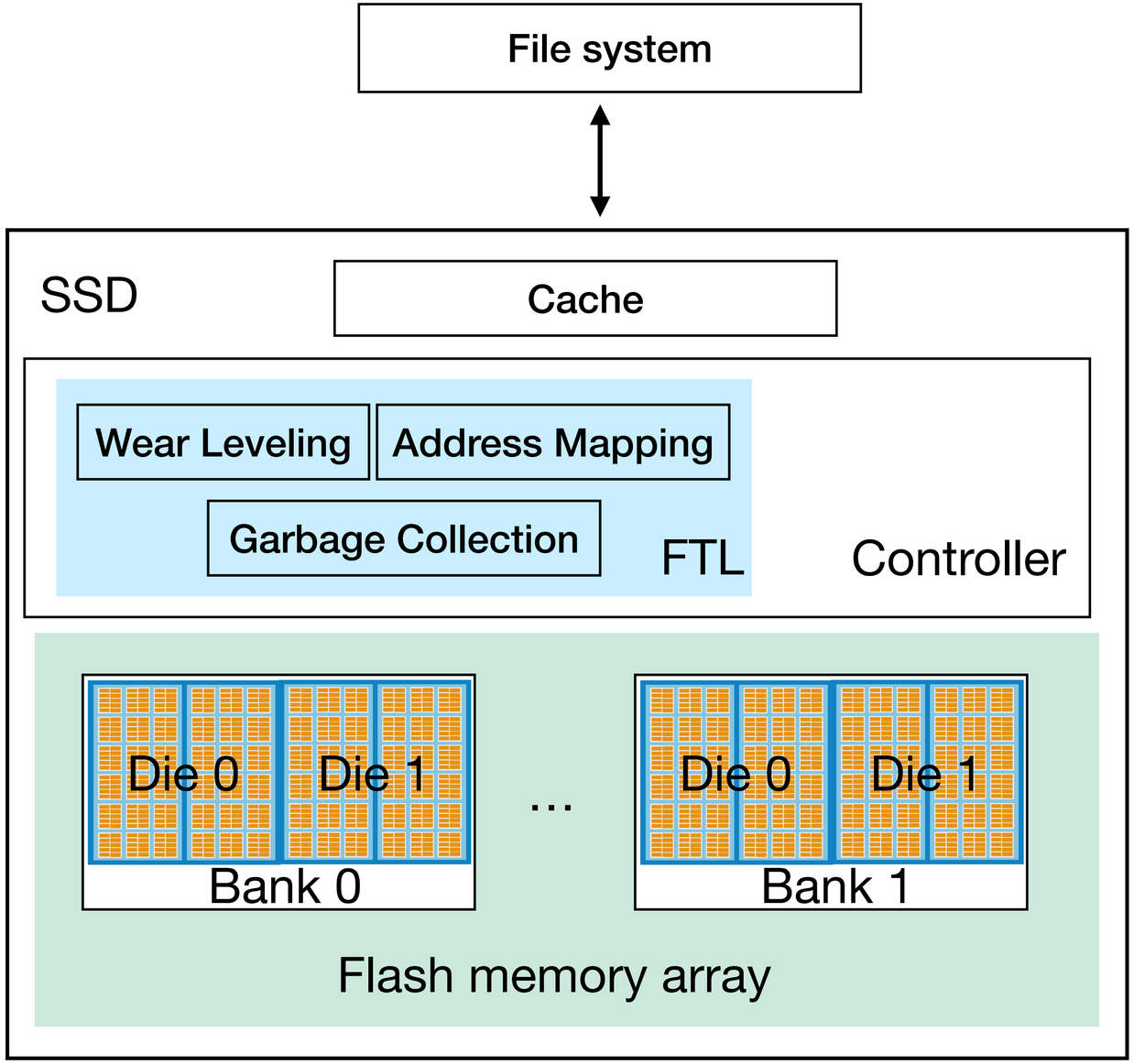
Flash-based SSD

- Flash memory array - flash chips that store data
  - For example, 1 TB SSD will have
    - 64 banks
    - Bank - 16 GB (2 dies)
    - Die - 8 GB (2 planes)
    - Plane - 4 GB (4096 blocks)
    - Block - 1 MB (128 pages)
    - Page - 8 KB (~10K TLC cells)
    - TLC - a single transistor holding 3 bits



Flash-based SSD

- **Cache** - volatile memory
  - caching/buffering data
  - mapping tables
- **Controller** - manages data and communicates with file system
  - **Flash Translation Layer (FTL)**
    - address mapping
    - wear leveling
    - garbage collection



SSD problems

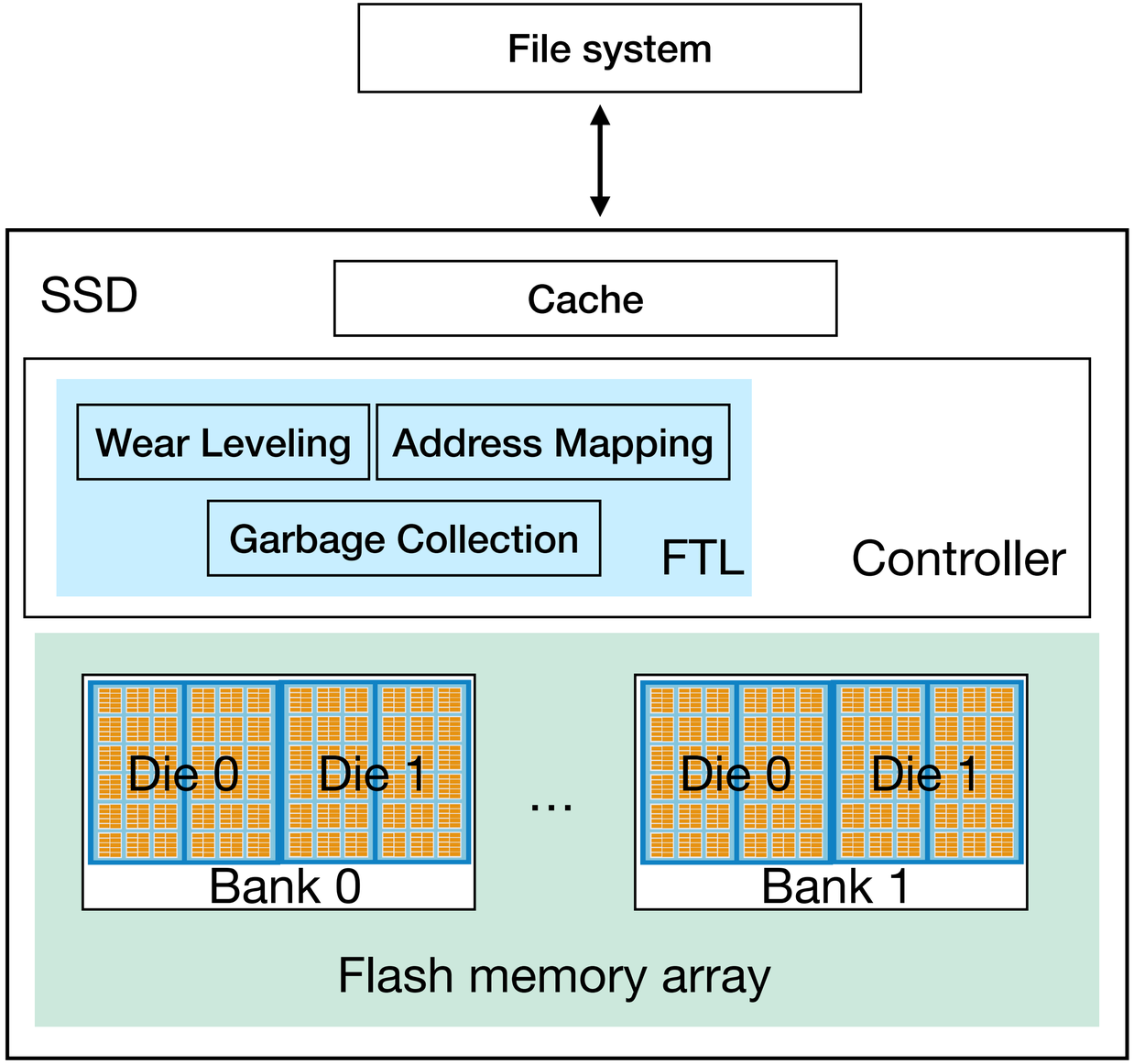
- **Write amplification** - when physical writes exceed logical writes
  - logical write - issue a write request from the OS
  - physical write - what ends up being (re)written on the SSD
  - e.g. OS requests to write 4 KB of data but wear leveling and garbage collection results in 20 KB of physical writes
- **Read/write disturbance** - programming/re-programming cells can flip bits in neighbouring cells
  - reduced by programming pages in erased block in order (low to high)
- **Cell-to-cell interference** - reading/writing can flip bits in neighbouring cells
  - reduced by error correction algorithms

FTL goals

- **Increase parallelism** - write to multiple flash chips in parallel
- **Reduce write amplification**
- **Improve wear leveling**
  - single blocks should not be programmed/erased too often
- **Reduce errors** from disturbances and interferences

FTL

- Address mapping
  - translate **logical blocks** to read/erase/program ops on **physical blocks** and **physical pages**
- Garbage collection
- Wear leveling



Address mapping

- **Direct mapping**
  - reading from logical page N -> reading from physical page N
  - writing to logical page N
    - move N's physical block B to a new location B1
    - erase physical block B
    - write the new page N
    - copy back old pages from B1 back to B
- **Performance**
  - **speed** - reading is OK, but writing will be slower than HD due to write amplification
  - **reliability** - not OK
    - some blocks may be repeatedly overwritten
    - OS controls writing but is not aware of the SDD block status

reading is OK

write amplification

writes are very slow

Address mapping

- **Log-structured FTL**
  - write to logical block N
    - write to the next free spot
  - reading from logical block N
    - a mapping table with physical-to-logical address translations

Address mapping

Another toy example

OS sees the SSD as a regular drive with N bytes of storage

OS writes/reads in chunks of 4KB

Logical addresses are given to logical blocks

SSD translates them to its physical pages

and stores translations in the mapping table

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	4KB	4KB	4KB	4KB	4KB	4KB	4KB	4KB	4KB	4KB	4KB	4KB
state	i	i	i	i	i	i	i	i	i	i	i	i

Logical block	Physical page

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data												
state	i	i	i	i	i	i	i	i	i	i	i	i

OS

write a1 to logical address 100

SSD

find the lowest invalid/empty block

Logical block	Physical page

page is 4KB

block is 16KB

SSD programs pages and erases blocks

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data												
state	e	e	e	e	i	i	i	i	i	i	i	i

OS

write a1 to logical address 100

SSD

find the lowest invalid/empty block

erase block 0

Logical block	Physical page

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1											
state	v	e	e	e	i	i	i	i	i	i	i	i

OS

write a1 to logical address 100

SSD

find the lowest invalid/empty block

erase block 0 program page 00

Logical block	Physical page

page is 4KB

block is 16KB SSD programs pages and erases blocks

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1											
state	v	e	e	e	i	i	i	i	i	i	i	i

Logical block	Physical page
100	0

OS

write a1 to logical address 100

SSD

find the lowest invalid/empty block

erase block 0 program page 00

save the mapping

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2										
state	v	v	e	e	i	i	i	i	i	i	i	i

Logical block	Physical page
100	00
101	01

OS

write a2 to logical address 101

SSD

block 0 is erased and there is an empty page there

program page 01

save the mapping

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1									
state	v	v	v	e	i	i	i	i	i	i	i	i

Logical block	Physical page
100	00
101	01

OS

write b1 to logical address 2000

SSD

block 0 is erased and there is an empty page there

program page 02

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1									
state	v	v	v	e	i	i	i	i	i	i	i	i

Logical block	Physical page
100	00
101	01
2000	02

OS

write b1 to logical address 2000

SSD

block 0 is erased and there is an empty page there

program page 02

save the mapping

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2								
state	v	v	v	v	i	i	i	i	i	i	i	i

OS

write b2 to logical address 2001

SSD

block 0 is erased and there is an empty page there

program page 03

Logical block	Physical page
100	00
101	01
2000	02

page is 4KB

block is 16KB

Address mapping

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2								
state	v	v	v	v	i	i	i	i	i	i	i	i

OS

write b2 to logical address 2001

SSD

block 0 is erased and there is an empty page there

program page 03

save the mapping

Logical block	Physical page
100	00
101	01
2000	02
2001	03

page is 4KB

block is 16KB

Log-structured write

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2								
state	v	v	v	v	i	i	i	i	i	i	i	i

Logical block	Physical page
100	00
101	01
2000	02
2001	03

OS

write c1 to logical address 100

SSD

erase block 1

move contents of block 0 there

write c1

write a2, b1, b2

update all mappings

what is this an example of?

write amplification

page is 4KB

block is 16KB

SSD programs pages and erases blocks

Log-structured write

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2								
state	v	v	v	v	e	e	e	e	i	i	i	i

OS

write c1 to logical address 100

SSD

erase block 1

Logical block	Physical page
100	00
101	01
2000	02
2001	03

page is 4KB

block is 16KB

Log-structured write

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2	c1							
state	v	v	v	v	v	e	e	e	i	i	i	i

OS

write c1 to logical address 100

SSD

erase block 1

write c1 to page 04

update the table

Logical block	Physical page
100	04
101	01
2000	02
2001	03

page is 4KB

block is 16KB

Log-structured write

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2	c1	c2						
state	v	v	v	v	v	v	e	e	i	i	i	i

OS

write c2 to logical address 101

SSD

write c2 to page 05

update the table

Logical block	Physical page
100	04
101	05
2000	02
2001	03

page is 4KB

block is 16KB

SSD programs pages and erases blocks

Log-structured FTL

- **Write to logical block N**
  - write to the next free spot
- **Reading from logical block N**
  - use a mapping table with logical-to-physical address translations
- **No in-place writing** (just like in the Log-structured file system LFS)
  - **avoids write amplification**
    - blocks are erased once in a while
  - **better wear leveling**
    - writes are distributed across the drive
  - **less read/write disturbance**
    - pages are always written in order

Garbage collection

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	a1	a2	b1	b2	**c1**	**c2**						
state	v	v	v	v	v	v	e	e	i	i	i	i

Logical block	Physical page
100	**04**
101	**05**
2000	02
2001	03

OS

overrode a1 and a2 with c1 and c2

SSD

updated the mapping table

page is 4KB

block is 16KB

SSD programs pages and erases blocks

Garbage collection

Problem: what do we do with the dead blocks?

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	~~a1~~	~~a2~~	b1	b2	c1	c2						
state	v	v	v	v	v	v	e	e	i	i	i	i

a1 and a2 were overwritten and nothing points at them anymore

unless we do something, the space will be wasted

Logical block	Physical page
100	04
101	05
2000	02
2001	03

OS

overrode a1 and a2 with c1 and c2

SSD

updated the mapping table

page is 4KB

block is 16KB

SSD programs pages and erases blocks

Garbage collection

Problem: what do we do with the dead blocks?

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data	~~a1~~	~~a2~~	b1	b2	c1	c2						
state	v	v	v	v	v	v	e	e	i	i	i	i

check mapping table to find a block with dead pages -> block 0 has some

read live data from block 0

Logical block	Physical page
100	04
101	05
2000	02
2001	03

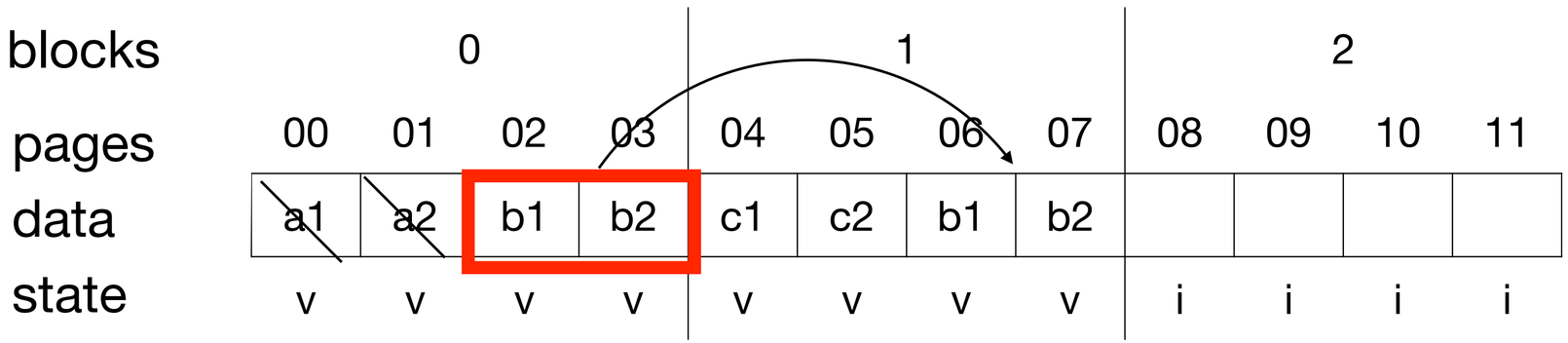
page is 4KB

block is 16KB

SSD programs pages and erases blocks

Garbage collection

Problem: what do we do with the dead blocks?



Logical block	Physical page
100	04
101	05
2000	02
2001	03

check mapping table to find a block with dead pages -> block 0 has some

read live data from block 0

write live data to the end of the log

page is 4KB

block is 16KB

Garbage collection

Problem: what do we do with the dead blocks?

blocks	0				1				2			
pages	00	01	02	03	04	05	06	07	08	09	10	11
data					c1	c2	b1	b2				
state	e	e	e	e	v	v	v	v	i	i	i	i

check mapping table to find a block with dead pages -> block 0 has some

read live data from block 0

write live data to the end of the log

erase block 0 for reuse

Logical block	Physical page
100	04
101	05
2000	02
2001	03

page is 4KB

block is 16KB

Garbage collection

- Find a block with many dead pages (ideally all of them)
- Write live pages (if any) to the unused space
- Free the block with old data
- This takes time and SSD space
- Solution: **overprovision**
  - manufacturers add extra capacity (5-10%) just for garbage collection
    - only the SSD controller can write to it, OS has no access
  - offsets space and bandwidth consumed by garbage collection
    - e.g. less frequent GC will require more space to store garbage and can be done when the device is not used heavily

Mapping table

- How much storage is needed for the mapping table?
  - num_entries * entry_size
  - e.g. for 1TB hard drive, 4KB pages, and 4 byte entries
    - 250M entries * 4 bytes = 1GB for just storing the mapping table
    - also will be expensive to search it
- What has memory virtualization taught us?
  - How did we reduce the size of the page tables?

size of the mapping table

num_entries * entry_size

Logical block	Physical page
100	04
101	05
2000	02
2001	03

Mapping table

- Problem: page-mapped tables require **too much memory**
- **Solution #1**: group reads/writes into **larger blocks**
  - pros: smaller tables
  - cons: write amplification
- **Solution #2**: caching - maintain a **cache** of the FTL in RAM
  - reads are more efficient
  - modified mappings will need to be written to the disk
  - cons: need a good replacement policy for FTL entries

Wear leveling

- Problem #1: Erasing and programming the same pages **wears them out**
  - Need to evenly distribute operations across the SSD
- Solution: **Log-structured FTL**
  - no in-place writes
  - all writes are to a new location on the hard drive
- Problem #2: blocks with long-lived data **do not get overwritten**
- Solution: FTL reads long-lived data and rewrites it somewhere else

SSD vs HDD

	Device	Random		Sequential	
		Reads (MB/s)	Writes (MB/s)	Reads (MB/s)	Writes (MB/s)
0.15-0.5 $/GB	SSD	**200-500**	**100-400**	**500-1000**	**400-1000**
0.02-0.05 $/GB	HDD	0.5-2	0.5-1	100-200	80-160

- SSD is **a lot faster** on random workload
  - writes are fast due to log-structured design - turns random writes into sequential
- SSDs are **faster** than HDDs on sequential workloads but the gap is smaller
- HDDs are still **a lot cheaper** and **more reliable** in some aspects

Summary

- **Log-structured file system**
  - the whole file system is one **big log**
    - **no in-place writes**
    - updates are **buffered** in RAM segments
    - written to a new location every **5-30 sec**
    - **garbage collection** cleans up "dead" data blocks
- Flash-based SSD
  - **no moving parts** but other limitations
    - **cannot write in-place** -> work well with a log-structured approach
    - repeated writing physically **wears out** cells -> need to distribute wear

Reading

- Chapter 43 Log-structured File System (LFS)
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-lfs.pdf
- Chapter 44
  - https://pages.cs.wisc.edu/~remzi/OSTEP/file-ssd.pdf

THE END

Exam

- **AC 122 A**
- **Saturday April 11**
- **2:30 - 4:30 PM**
- Do not forget to **bring your student id**

- Format will be the same as midterm
  - 60% MCQ - 30 questions 2 points each
  - 40% short answers - 8 questions 5 points each
- Topics covered
  - ~30% pre-midterm - virtualization of CPU and memory
  - ~70% post-midterm - concurrency and persistence

Exam prep

- See tips from the last lecture
  - Use your time well
    - Aim to review the class material evenly
- I will post another mega-quiz over the weekend

Office hours

- Tuesday April 7
  - Will be held on Monday 12:00-13:00 PM
- Thursday April 9 13:00-14:00 PM

THE LAST QUIZ 20

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999